Josefines bokregister

Att skapa en konsolapplikation i C#.NET

Josefine Backlund

MITTUNIVERSITETET
Institutionen för Informations-system och teknologi

Författare: Josefine Backlund, joba1602@student.miun.se
Utbildningsprogram: TWEUG, Webbutveckling, 120 hp
Huvudområde: Datateknik
Termin, år: HT, 2026

Josefines bokregister – att skapa en konsolapplikation i C#.NET
Josefine Backlund 2026-10-28

Sammanfattning

Denna rapport beskriver processen som tagit fram en konsolapplikation i C#.NET som agerar som ett register över böcker användaren har läst, läser eller ska läsa. Användaren kan lägga till, ändra, ta bort, filtrera och visa lagrade böcker samt justera lässtatus på dessa. Applikationen skapades med hjälp av Visual Studio. Slutsatsen beskriver vad som gått bra och dåligt under processen, och vad som hade kunnat göras annorlunda.

Innehållsförteckning

Innehållsförteckning **3**
1 Introduktion **1**
1.1 Bakgrund och problemmotivering 1
1.2 Övergripande syfte 1
1.3 Detaljerad problemformulering 1
2 Teori **2**
2.1 C#.NET 2
2.2 Visual Studio 2
3 Metod **3**
4 Konstruktion **4**
5 Resultat **8**
6 Slutsatser **9**

Josefines bokregister – att skapa en konsolapplikation i C#.NET
Josefine Backlund 2026-10-28

1 Introduktion

1.1 Bakgrund och problemmotivering

Om man gillar att läsa böcker kan man snabbt få en rätt så lång lista på potentiella böcker man vill läsa, och det kan vara svårt att hålla koll på hur många man har läst. Man kan såklart skriva en egen lista eller bara hålla informationen i huvudet gällande detta ämne, men med det valet finns alltid risken att listan kan komma bort eller att man till exempel glömmer bort en bok man är intresserad av att läsa. Med hjälp av en egen applikation för detta kan man lagra informationen digitalt, men det finns många olika aspekter av en applikation man måste ta hänsyn till för ett långvarigt och stabilt program. Informationen som ska lagras behöver en struktur, lagringen behöver en metod, applikationen behöver lämpliga funktioner för att hantera lagringen och felaktig inmatning måste hanteras. Att skapa en applikation från grunden ger utrymme att nyttja programmeringskunskaper i praktiken, och C#.NET erbjuder denna möjlighet.

1.2 Övergripande syfte

Syftet med detta projekt är att dokumentera processen som skapat en konsolapplikation med hjälp av C#.NET som gör det möjligt för användaren att hantera ett register med böcker.

1.3 Detaljerad problemformulering

Målet med detta projekt är att skapa en applikation som erbjuder en funktionalitet som gör att användaren kan lägga till och därmed spara böcker som också kan hanteras, samt få en överblick över böcker som blivit lästa och inte.

Användaren ska kunna:

- Lägga till en ny bok med titel, författare, genre, år och lässtatus.
- Ändra eller ta bort böcker.
- Visa alla böcker.
- Söka bland registrerade böcker med hjälp av titel eller författare.
- Markera bok som oläst, pågående eller utläst.

2 Teori

2.1 C#.NET

C# är ett programmeringsspråk som kan köras i ramverket *.NET*, och är utvecklat av Microsoft. Språket är objektorienterat, och delar likheter med språken Java och C++. Att språket är objektorienterat innebär att koden har en struktur som gör det möjligt att återanvända mycket av koden med hjälp av *klasser* [1]. Klasser kan beskrivas som en förutbestämd struktur som olika instanser, alltså objekt, ska följa. Dessa objekt erhåller i sin tur alla egenskaper och metoder som klassen som skapat dem har [2]. En annan fördel med objektorienterad programmering är att det finns sätt att göra koden säkrare med hjälp av *interfaces*. Interfaces är som en klass, men kan inte instansiera något. Interfaces har till exempel inte några fält, och de kan inte användas för att skapa ett objekt. De kan dock innehålla tomma metoder som andra klasser kan implementera med ett ":". Syftet med interfaces är att visa de viktigaste delarna av ett objekt, vilket gör att ett objekt inte kommer köras om metoderna från ett implementerat interface saknas [3].

En annan fördel med just C# är att något vid namn *LINQ* (Language Integrated Query) finns tillgängligt vid utvecklingen. LINQ är integrerat direkt i programmeringsspråket, vilket gör det möjligt att efterfråga viss data i samlingar, som till exempel listor. Syntaxen för detta påminner om SQL-programmering. Med hjälp av detta kan man till exempel filtrera, sortera och gruppera data baserade på deklarerade villkor [4].

2.2 Visual Studio

Visual Studio är en samlad utvecklingsmiljö som utvecklats av Microsoft där det är möjligt att bland annat skapa konsol-, desktop- och mobilapplikationer. Dessa kan byggas, testas och lanseras på ett och samma ställe. Miljön erbjuder många olika språk att använda sig av, till exempel C#, Python och JavaScript m.m [5].

3 Metod

Processen för att skapa applikationen kommer att delas in i olika steg. Eftersom att detta är en konsolapplikation kommer inte mycket behöva göras när det kommer till design, men för att visualisera alla funktioner som krävs så kommer vanliga utskrifter i konsol göras och sparas för att demonstrera vilka funktioner som kommer att finnas och hur dessa ska förmedlas till användaren.

När applikationen har en etablerad plan så kommer ett nytt projekt startas med hjälp av Visual Studio 2026. En klass kommer att skapas för att etablera strukturen på den lagrade informationen, alltså böckerna. En annan kommer att beskriva de funktioner som bokregistret ska ha, alltså hur man kan manipulera den lagrade informationen. Det kommer även finnas ett interface som beskriver hur lagringen ska gå till och vilka funktioner som behövs. Till sist behövs det också en klass som implementerar lagringsfunktionerna enligt skapat interface. Alla dessa klasser kommer sedan att komma till användning i “Program.cs”-filen där applikationen blir till.

För att uppfylla de mål som krävs för att applikationen ska fungera som tänkt så kommer C# att användas som programmeringsspråk tillsammans med ramverket .NET i Visual Studio 2026. För att användaren ska kunna spara, ändra eller ta bort böcker så kommer en JSON-fil skapas i syfte för förvaring så att alla förändringar sparas oavsett om applikationen är igång eller inte.

4 Konstruktion

Ett nytt projekt startades i Visual Studio 2026, och för att kunna ha mer kontroll, överblick över framfart och av säkerhetsskäl så skapades ett repo länkat till Github. Till att börja med så skapades utskrifter i konsolen som skulle representera utseendet av applikationen och dess funktioner, till exempel hur det kommer att se ut för användaren i menyn eller hur processen för att lägga till en bok kommer att se ut (se bilaga A).

Efter utskrifterna gjordes så skapades nödvändiga klasser som erhåller de grundläggande funktionerna som behövs för att skapa applikationen. Olika klasser skapades i syfte att få mer kontroll och struktur över koden, vilket i sin tur gör det lättare för utomstående parter att granska funktionalitet och om koden behöver ändras eller uppdateras kommer det vara enklare att hitta i filerna.

Först skapades klassen “Book.cs”, som erhöll de egenskaper varje lagrad bok ska ha, alltså: titel, författare, genre, år och lässtatus. Sedan skapades ett interface, “IBookRegisterStorage.cs”, som beskriver funktioner som ska hämta lagrade böcker, och spara ändringar i lagringen. Efter det skapades “JsonBookRegisterStorage.cs”, som implementerar funktionerna från interfacet specifikt i syfte att skapa en JSON-fil och manipulera denna med sagda funktioner. Slutligen så skapades “BookRegister.cs” som erhåller de funktioner användaren behöver för att kunna manipulera lagringen, alltså: lägga till böcker, ändra böcker, filtrera böcker, ta bort böcker, justera lässtatus och även att hämta en lista med lagrade böcker. Vid filtreringen av böcker lämpade sig LINQ-metoden “Where” för att returnera en ny lista med innehåll baserat på användarens input.

När lagringsmöjligheter och funktioner etablerats så implementerades dessa i “Program.cs”-filen. En huvudmeny med tillgängliga val för användaren skapades där valen kan göras genom att trycka på specifika tangenter på tangentbordet. Här ser användaren att hen kan lägga till, visa, söka efter, ändra, filtrera och ta bort böcker, justera lässtatus och avsluta programmet. Eftersom funktionerna redan fanns till hands att använda, så behövdes bara användarens input-instanser skapas och användas med rätt funktion för att få önskad effekt, se Figur 1.1.



Figur 1.1 Implementering av funktion

När alla funktioner kommit på plats och gör vad som förväntas var det dags att implementera felhantering för användaren, så att förväntad lagring ser ut som tänkt. Felhanteringen för strängar har en återkommande struktur, där steget för input-lagring är omringat av en while-loop som körs medan input är null. Användaren uppmanas att skriva typ av input, och kommer att fastna i en if-sats i loopen om input är null, vilket även får ett bool-värde att bli sant, vilket i sin tur triggar ett felmeddelande för användaren. Denna loop upphör när input har ett värde. Se Figur 1.2.



```
error = false;
while (String.IsNullOrEmpty(input))
{
    if (error)
    {
        RecurrentTopMenu();
        Console.WriteLine("Ange korrekt input!\n");
    }
    Console.WriteLine("Önskad input:");
    input = Console.ReadLine();
    if (String.IsNullOrEmpty(input))
    {
        error = true;
    }
}
```

Figur 1.2 Felhantering av sträng

Felhantering av typen nummer hanteras på ett liknande sätt, men körs medan ett boolvärde är sant, och detta värde ändras till falskt manuellt om inputen passerar if-satser som kontrollerar avsaknad input, men även om inputen går att omvandla till en siffra, så att användaren till exempel inte kan ange bokstäver som input, se Figur 1.3.



```
validateNumberInput = true;
while (validateNumberInput)
{
    if (error)
    {
        RecurrentTopMenu();
        Console.WriteLine("Ange önskad input! Använd önskad typ.\n");
    }
    Console.WriteLine("Input:");
    input = Console.ReadLine();
    if (String.IsNullOrEmpty(input))
    {
        error = true;
        continue;
    }
    else if (!Int32.TryParse(input, out int number))
    {
        error = true;
        continue;
    }
    validateNumberInput = false;
}
```

Figur 1.3 Felhantering av nummer

Efter all felhantering implementerats så har input under alla instanser testats genom att först försöka ange tomma värden, och sedan vid omständigheter där det är relevant ange fel typ av input, till exempel genom att ange en bokstav när man ska ange det index man vill förändra eller när man försöker ange ett index som inte existerar.

5 Resultat

En konsolapplikation har skapats där användaren kan lagra böcker och manipulera dessa på olika sätt. Användaren får olika möjligheter i menyn när applikationen har startats. Det är möjligt att lägga till en bok, där titel, författare, genre, år och lässtatus behöver anges. Efter att ha lagt till böcker är det möjligt för användaren att ändra den lagrade informationen efter önskat index matats in. Innan användaren ändrar på en bok så visas en lista upp med lagrade böcker och index för dessa. Det är även möjligt att ta bort lagrade böcker med hjälp av angivet index. Om användaren önskar se alla böcker som lagrats finns det även ett menyalternativ för detta. Med hjälp av sökord baserat på titel eller författare kan också användaren få ett sökresultat baserat på angiven input. Slutligen kan användaren också välja att endast justera lässtatusen på lagrade böcker, om användaren till exempel vill ändra en boks status från “pågående” till “utläst”.

6 Slutsatser

Att genomföra ett projekt med hjälp av C#.NET har gett mig både positiva och negativa upplevelser, men oavsett vad så har det åtminstone inneburit utveckling inom ett outforskat område. Att skriva i ett mer typat språk jämfört med tidigare upplevelser har varit intressant och lärorikt, och desto större projekt jag skapar, vilket innebär mer kod, desto större behov av struktur finns, vilket detta språk erbjuder på ett enkelt sätt. Att ta sig an ett nytt programmeringsspråk är både roligt och frustrerande, eftersom jag kan använda mycket av kunskapen sedan tidigare, men det kan också vara irriterande när sättet man brukar jobba på plötsligt inte fungerar.

Applikationen gör det den ska göra, men det finns mycket man kan jobba vidare med för att förbättra den och göra den mer intressant. Detta projekt påverkades starkt av tidsbrist, vilket ledde till att områden som annars hade utforskats inte blev av. Applikationen hade sett mer intressant ut och fått ett mer unikt “user interface” om justering av konsolen hade utförts. Man hade till exempel kunnat installera något paket för detta som hade gjort det möjligt att skapa organiserade tabeller för lagring av böckerna. Något annat som kan göras bättre är att utveckla funktionerna i applikationen. Om man hade implementerat datum för påbörjad och slutförd läsning av en bok och hur många sidor varje bok hade, hade man kunnat skapa funktioner som erbjuder användaren någon slags diagnostik över sin läsning, så att de till exempel kan se hur många sidor hen har läst per år. Det fanns även planer på att manipulera färgen på utskrifterna, att till exempel ha olika färger beroende på lässtatusen, men även detta togs bort på grund av tidsbrist. Koden kan definitivt också förbättras, då den känns repetitiv på många ställen. Antingen hade någon ytterligare klass kunnat skapas enbart för felhantering, eller så hade fler lokala funktioner kunnat skapas i “Program.cs”-filen för att minska redundansen.

Något av det viktigaste jag lärt mig av att genomföra detta projekt är att felhantering är ett rätt så stort och viktigt område. Det finns så oerhört många potentiella fel en användare kan göra, och förutsätter man inte dessa som skapare så kan programmet bli rätt så instabilt. Felhanteringen blev en mycket större del av hela utförandet än vad jag trodde att den skulle vara. Applikationen har inte så många funktioner, men att felhantera varje del av applikationen tog tid, och man får ta många aspekter i åtanke.

Källförteckning

[1] W3Schools. “C# Introduction” [Internet]. C 2026. Hämtad 2026-10-28, från: https://www.w3schools.com/cs/cs_intro.php

[2] W3Schools. “C# OOP” [Internet]. C 2026. Hämtad 2026-10-28, från: https://www.w3schools.com/cs/cs_oop.php

[3] W3Schools. “C# Interface” [Internet]. C 2026. Hämtad 2026-10-28, från: https://www.w3schools.com/cs/cs_interface.php

[4] GeeksForGeeks. “LINQ (Language Integrated Query” [Internet]. Uppdaterad 2026-04-20. Hämtad 2026-10-28, från: https://www.geeksforgeeks.org/c-sharp/linq-language-integrated-query/

[5] Microsoft Ignite. “What is Visual Studio?” [Internet]. C 2026. Uppdaterad 2026-03-20. Hämtad 2026-10-28, från: https://learn.microsoft.com/en-us/visualstudio/get-started/visual-studio-ide?view=visualstudio

Bilagor

Bilaga A - Skärmbilder på planerad utskrift

```
J O S E F I N E ' S  B O K R E G I S T E R


1. Lägg till ny bok
2. Visa alla böcker
3. Sök efter bok
4. Ändra en bok
5. Ta bort en bok
6. Justera lässtatus
X. Avsluta
```

```
J O S E F I N E ' S  B O K R E G I S T E R


Lägg till ny bok

Titel:

Författare:

Genre:

År:
```

```
J O S E F I N E ' S  B O K R E G I S T E R

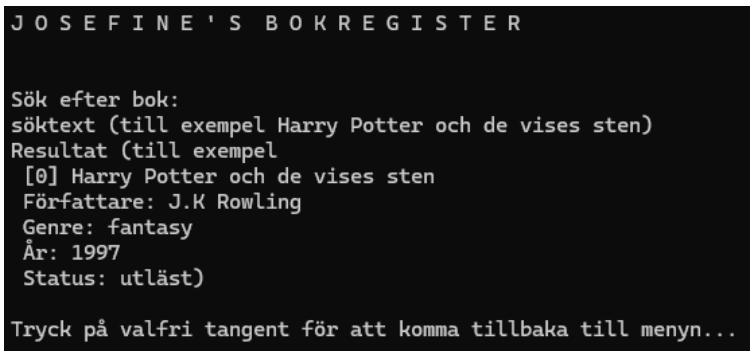

X. Tillbaka till meny

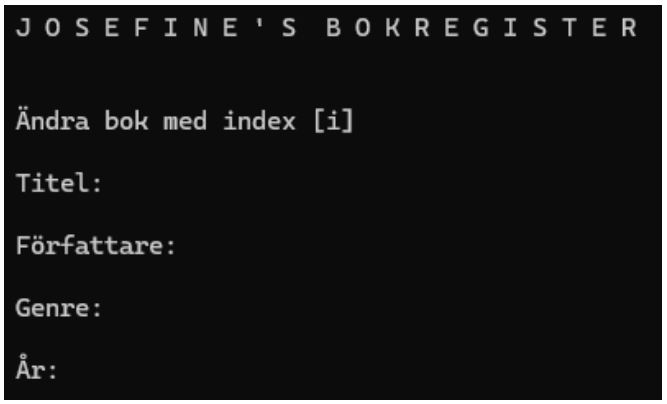
Mina böcker

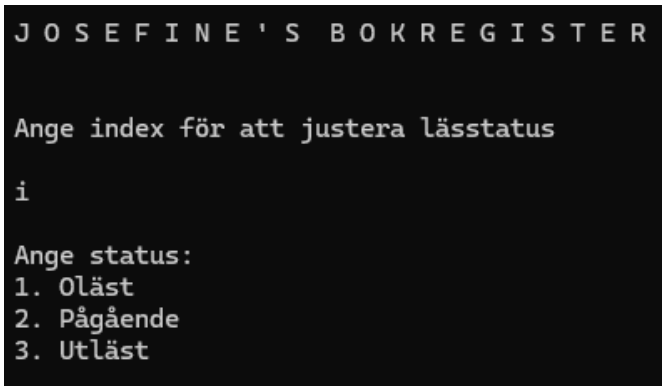
[0] Harry Potter och de vises sten
 Författare: J.K Rowling
 Genre: fantasy
 År: 1997
 Status: utläst

[1] Harry Potter och hemligheternas kammare
 Författare: J.K Rowling
 Genre: fantasy
 År: 1998
 Status: pågående

[2] Harry Potter och fången från Azkaban
 Författare: J.K Rowling
 Genre: fantasy
 År: 1999
 Status: oläst
```


J O S E F I N E ' S B O K R E G I S T E R
Sök efter bok:
söktext (till exempel Harry Potter och de vises sten)
Resultat (till exempel
[0] Harry Potter och de vises sten
Författare: J.K Rowling
Genre: fantasy
År: 1997
Status: utläst)
Tryck på valfri tangent för att komma tillbaka till menyn...


J O S E F I N E ' S B O K R E G I S T E R
Ändra bok med index [i]
Titel:
Författare:
Genre:
År:


J O S E F I N E ' S B O K R E G I S T E R
Ange index för att justera lässtatus
i
Ange status:
1. Oläst
2. Pågående
3. Utläst